

Final Year Project Report AI-Powered PDF Assistant

Project Title: AI-Powered PDF Assistant — Summarize, Query, and Convert PDFs

Student Name: ____

Roll No / Reg No: ____

Department: ____

College / Institute: ____

Guide / Mentor: ____ Year

/ Batch: 2025

Abstract

The **AI-Powered PDF Assistant** is a comprehensive system designed to transform static PDF documents into dynamic, interactive knowledge resources. It provides semantic search and question-answering capabilities, advanced summarization, and audio narration. The assistant first **extracts text** from uploaded PDFs, then uses **smart chunking** to preserve context within manageable sections. Each chunk is embedded into dense vectors via a Sentence-BERT model, enabling **Retrieval-Augmented Generation (RAG)** for downstream tasks. When a user asks a question or requests a summary, the system retrieves the most relevant chunks and passes them to a Large Language Model (LLM) that generates concise answers or summaries. Additional features include automatic **chapter extraction**, **exam question generation**, and **text-to-speech (TTS)** audio conversion of the PDF content. These functionalities make the assistant a powerful tool for students and professionals to quickly access and consume information: for instance, one can chat with a PDF or get key takeaways in seconds, rather than manually reading the entire document.

This report details the design and implementation of the system, covering the end-to-end data flow, core algorithms (chunking, embedding, retrieval, generation), and practical considerations such as token efficiency and deployment.

Keywords: PDF summarization, RAG, embeddings, sentence-transformers, Flask, PDF-to-Audio, gTTS, text chunking, document QA

Table of Contents

1. [Introduction](#)
2. [Problem Statement](#)
3. [Objectives](#)
4. [Scope](#)
5. [Literature Survey](#)
6. [System Requirements](#)
7. [System Design and Architecture](#)
 - 7.1 [High-level Architecture](#)
 - 7.2 [Sequence Flow \(Upload Summary\)](#)
8. [Modules and Components](#)
 - 8.1 [Upload & Storage](#)

- 8.2 [Text Extraction](#)
 - 8.3 [Chunking](#)
 - 8.4 [Embedding](#)
 - 8.5 [Retrieval](#)
 - 8.6 [LLM Interaction](#)
 - 8.7 [Audio and Document Export](#)
 - 8.8 [Session Store](#)
 - 9. [Algorithms and Data Flow](#)
 - 9.1 [Chunking Algorithm](#)
 - 9.2 [Embedding + Retrieval](#)
 - 9.3 [Summarization Pipeline \(Two-step\)](#)
 - 9.4 [Question-Answering \(RAG\)](#)
 - 10. [Implementation Details](#)
 - 11. [API Endpoints](#)
 - 12. [Testing and Results](#)
 - 13. [Deployment and Installation](#)
 - 14. [Security Considerations](#)
 - 15. [Limitations](#)
 - 16. [Future Work](#)
 - 17. [Conclusion](#)
 - 18. [References](#)
 - 19. [Appendix](#)
-

1. Introduction

The proliferation of digital documents has made efficient information access increasingly important. Students, researchers, and professionals often encounter lengthy PDFs (textbooks, research papers, reports) from which they need specific insights or summaries. Manually reading through entire documents is **time-consuming** and can miss key information, while simple keyword searches often fail to capture the full context of the content ^{3 1}.

Recent advances in artificial intelligence, especially in **large language models (LLMs)** and semantic search, enable the creation of tools that can “**chat with**” a document. An AI-powered PDF assistant can *scan a document’s content* and answer questions or produce summaries as if it were an expert that knows everything in the PDF ^{1 2}. For example, a user who receives dozens of client reports or legal contracts every week may simply ask, “What is the refund policy in this contract?” or “Summarize the findings in Section 3”, and receive accurate responses drawn directly from the document ¹.

This project implements such an assistant. It uses a **Retrieval-Augmented Generation (RAG)** approach: when a user queries the system, relevant segments of the PDF are retrieved via semantic search, then an LLM generates the final answer or summary. The system also provides **conversational chat with PDF content**, **PDF summarization**, **automated question generation**, **chapter/section extraction**, and **text-to-speech (TTS)** audio conversion. In sum, it transforms static PDF files into interactive knowledge bases, enabling quick insights and accessibility (e.g., converting textbook material into audio for learning on the go ^{5 4}). The design and implementation of the system are described in the following sections.

2. Problem Statement

The goal is to develop a **web-based application** that ingests PDF documents and provides various AI-powered outputs grounded in the PDF content. The system must:

- **Accept PDF uploads:** Allow users to upload one or more PDF files via a browser or API.
- **Extract and preprocess text reliably:** Use robust PDF text extraction that works on diverse layouts. Handle complications such as multi-column layouts or scanned pages (without embedded text) ⁶.
- **Produce semantic embeddings:** Convert the extracted text into vector embeddings that capture meaning (not just keywords) using a model like Sentence-BERT.
- **Chunk text content:** Split large documents into smaller “chunks” in a context-preserving manner, because LLMs have limited input lengths ⁷.
- **Support core functionalities:**
 - **Summarization:** Generate concise summaries of the entire PDF or sections in various styles (e.g., bullet points, paragraphs).
 - **Q&A:** Answer user questions with answers grounded in the PDF content.
 - **Chapter/Section detection:** Automatically identify and label chapters or sections if possible.
 - **Question generation:** Produce potential exam or discussion questions from the content.
 - **PDF-to-Audio conversion:** Convert the extracted text into spoken audio (MP3) for accessibility ⁴.
- **Interactive chat:** Maintain session-based chat where the user can ask follow-up questions about the PDF, with the system remembering context across turns.
- **Efficiency:** Minimize usage of expensive API tokens by compressing content (e.g., metasummarization of chunks).
- **Reliability:** Ensure accurate results by using context-aware retrieval rather than naive keyword matching.

For example, one challenge is that **scanned PDFs** (images of text) contain no extractable text, requiring OCR. The system should at least detect this limitation (PyPDF2 cannot read image-only PDFs ⁶). Another challenge is balancing chunk size: chunks must be large enough to preserve meaning but small enough to allow efficient retrieval and LLM processing ⁸ ⁷. The design addresses these challenges systematically as detailed below.

3. Objectives

This project has several key objectives:

- **Robust Text Extraction:** Use tools like PyPDF2 (or alternatives) to extract text from a wide variety of PDF layouts. Handle multi-column formats and note when pages contain only images (scanned text) ⁶.
- **Effective Chunking:** Develop strategies to break text into chunks of optimal size. We implement both word-based and overlapping character-based chunking to preserve context across splits ⁷ ⁸.
- **Semantic Embeddings:** Employ a compact Sentence-BERT model (specifically `all-MiniLM-L6-v2`) to generate dense 384-dimensional embeddings for each chunk. Sentence-BERT transforms text into embeddings that capture meaning ⁹ ¹⁰.

- **RAG-based Retrieval:** Implement Retrieval-Augmented Generation: encode user queries into vectors, perform semantic search over the chunk embeddings using cosine similarity (dot product), and retrieve the top-K relevant chunks.
- **LLM Integration for Generation:** Use a chat-capable LLM (via an API wrapper like Groq Client) to generate final answers and summaries. We design system and user prompts to guide the LLM appropriately (e.g., instructing it to answer only from the provided context).
- **User Interface & APIs:** Create a Flask-based web UI and REST API endpoints to allow users to upload PDFs, request summaries or Q&A, and chat with document context.
- **Audio and Document Export:** Integrate Google TTS (`gTTS`) to produce MP3 audio from extracted text ⁵. Enable downloading of results in formats like MP3, PDF, or Word.
- **Efficiency:** Reduce token usage by summarizing chunks before combining them (a metasummarization step). This two-step compression yields concise intermediate content for the final summary, saving API tokens.
- **Accessibility:** Ensure the assistant is usable on standard hardware and easy to deploy (e.g., via Docker).

These objectives aim to create a practical tool for academic and professional users, enabling them to quickly glean insights from lengthy documents through advanced AI techniques.

4. Scope

This system is designed primarily for academic and educational use cases, such as:

- **Textbooks and Lecture Notes:** Summarizing chapters, generating study questions, and providing audio versions of course materials.
- **Research Papers and Reports:** Answering specific queries (e.g., “What method was used in Experiment 2?”), extracting key points, and obtaining quick overviews.
- **Professional Documents:** For example, legal contracts or business reports, where users may want to pinpoint clauses or findings without manual reading.

Currently, the system supports **English** text. Extending to other languages would involve using equivalent embedding models or performing translation. The design is modular, so language support can be expanded by swapping models or adding translation pre-steps.

We do not focus on **OCR** of scanned PDFs in this version, as PyPDF2 (used here) only extracts digitally embedded text ⁶. Supporting scanned images would require an OCR engine like Tesseract, which is noted as future work.

Security and privacy aspects (like user accounts or saved document access control) are outside the immediate scope; we assume a trusted environment or a proof-of-concept setting.

5. Literature Survey

Our work builds on several research and engineering areas: semantic document retrieval, summarization with LLMs, text chunking, and accessibility via TTS. Key points from the literature include:

- **Retrieval-Augmented Generation (RAG):** RAG techniques combine a knowledge retrieval step with LLM generation ³. Instead of relying on the LLM’s static knowledge, RAG retrieves relevant text chunks from an external source to ground the response. This improves answer accuracy for content not in the model’s training data. For example, GPT systems enable RAG by embedding user questions and searching uploaded documents using semantic vector search

¹¹₃. This project uses a similar approach: user queries are converted to embeddings and matched to document chunks via cosine similarity ¹¹₉.

- **Sentence Embeddings and Dense Retrieval:** The Sentence-BERT (SBERT) family of models (Reimers & Gurevych, 2019) produces semantically meaningful sentence embeddings using a siamese BERT architecture ⁹. These embeddings allow fast semantic similarity search (cosine or dot-product) compared to naive BERT two-sentence comparisons. The popular SentenceTransformers library provides many pre-trained SBERT models ¹². We use the compact all-MiniLM-L6-v2 model, which yields 384-dimensional vectors ¹⁰. Dense retrieval using SBERT is well-suited for limited-scale document search.

- **Chunking Strategies:** Documents must be split into chunks for embedding. LLMs have **limited context windows**, so chunking is essential ⁷ ⁸. Good chunking preserves semantic coherence: too large chunks mix topics (hurting retrieval accuracy), while too small chunks lose context. Weaviate’s analysis emphasizes that chunking is a critical factor in RAG performance ⁷ ⁸. They note that chunking should balance retrieval precision and context preservation. Our implementation uses word-based and overlapping character-based chunking to strike this balance ⁷ ⁸.

- **Summarization with LLMs:** Modern summarization often uses LLM prompts. A multi-stage or “mapreduce” approach can improve results on long texts. For instance, one can summarize individual chunks and then combine them into a final summary ¹³. Our two-step summarization first generates concise bullet summaries of each chunk, then feeds these into the LLM to produce a coherent overall summary. This approach echoes strategies for handling long documents with limited context (similar ideas were explored in legal document summarization ¹³ and other domains).

- **Automatic Question Generation (AQG):** Generating questions from text is a well-studied task in NLP. AQG systems create meaningful questions based on a given passage ¹⁴. They have applications in education (e.g., generating practice quiz questions) and chatbots. For example, Mulla & Gharpure (2023) review methods for automatic question generation and their use in reading comprehension tasks ¹⁴. We incorporate a simple question-generation feature, leveraging the LLM to output plausible questions from the content, following these principles.

- **Accessibility via Text-to-Speech (TTS):** Audio narration is important for accessibility. TTS tools convert text to spoken words. Studies have shown TTS can aid reading comprehension, especially for learners or users with disabilities ⁴. Our system uses the open-source Google TTS (gTTS) to generate MP3 audio files of the PDF’s text ⁵. This aligns with findings that “presenting reading material orally in addition to text can help users better comprehend written content” ⁴.

- **Related Tools:** Products like Smallpdf’s AI Assistant demonstrate the demand for such features. Smallpdf advertises that their AI PDF chat can “provide summaries of lengthy PDFs” and answer specific document questions ². Similarly, custom projects (e.g., Suleman Safdar’s Medium post) describe pipelines with PDF parsing, chunking, vector search, and LLM Q&A ¹. We draw on these best practices in our system design.

In summary, the literature underscores the effectiveness of combining **semantic embeddings, dense retrieval, and LLM generation** for document understanding. Chunking is a critical preparation step ⁷ ⁸, and TTS adds significant educational value ⁴ ⁵. The next sections describe how we implement and extend these ideas.

6. System Requirements

6.1 Software Requirements

- **Python (3.9+)**: The core language for development.
- **Flask**: A lightweight Python web framework for building the frontend and API endpoints ¹⁵. Flask is designed to be easy to get started with yet scalable for complex apps.
- **PyPDF2**: For extracting text from PDF files. It handles digital text content but not scanned images ⁶.
- **sentence-transformers**: Provides pre-trained Sentence-BERT models (`all-MiniLM-L6-v2`) for embedding text chunks.
- **NumPy**: For numerical operations on embedding vectors (e.g., dot-product similarity).
- **Groq or REST API client**: To send conversation/summarization prompts to the chosen LLM. (The example code uses a hypothetical `groq-client` ; in practice this could be any hosted LLM API.)
- **gTTS (Google Text-to-Speech)**: For converting text to spoken audio (MP3). gTTS interfaces with Google Translate's TTS APIs.
- **ReportLab**: If generating custom PDFs for output (e.g., writing audio transcripts or formatted results).
- **python-docx**: For exporting results to Word documents if needed.
- **Pillow (PIL)**: For any image handling (e.g., generating dynamic charts, although not heavily used here).

Additional libraries might include `flask-cors` , `dotenv` (for environment variable management), and others for utilities.

6.2 Hardware Requirements

- **Development**: A typical modern laptop or desktop with at least 8 GB of RAM. The embedding and summarization pipeline is more CPU-bound (no large local GPU needed unless the LLM is run locally).
- **Production**: For moderate use, 2+ CPU cores and 4 GB+ RAM should suffice. If scaling to large numbers of users or running a local LLM, additional resources or a GPU may be required. The current design uses a hosted LLM API (e.g., Groq or OpenAI), so GPU is not strictly necessary. Persistent storage (disk space) is needed for uploaded PDFs and generated outputs.

7. System Design and Architecture

7.1 High-level Architecture

The system follows a modular web-service architecture:

- **Frontend (Flask Templates)**: A simple HTML/CSS/JS interface where users can upload PDFs, select tasks (summarize, ask questions, chat, audio), and view results. E.g., forms for PDF upload, chat interface for Q&A.
- **Backend (Flask App)**: Handles HTTP requests. Key backend components:
- **Upload Handler**: Receives PDF files (via routes like `/ai-summarize` , `/chat-pdf-upload`), saves them temporarily, and triggers processing.
- **Text Extraction Module**: Reads the PDF using PyPDF2, extracts text from each page.
- **Chunking Module**: Splits the full text into chunks (either word-based or fixed-size overlapping windows) ⁷.
- **Embedding Module**: Uses `sentence-transformers` to encode chunks into numeric vectors (dense embeddings).

- **Vector Store:** In-memory per-session storage of chunk embeddings and text. (For scale, could swap in a vector DB like Milvus or Pinecone.)
- **Retrieval Module:** On queries, embeds the query text, computes similarity (dot-product) with stored vectors, and selects the top-K matching chunks.
- **LLM Interface:** A wrapper (e.g., `call_groq_chat`) that sends prompts to the LLM API and returns the generated answer or summary.
- **Summarization Pipeline:** Runs a two-step pipeline where each chunk is first summarized (possibly with bullet compression), then a meta-summary is generated.
- **Audio Generation:** Uses gTTS to convert the entire document text or sections to an MP3 file.
- **Export Module:** Optionally generates downloadable outputs (DOCX, PDF) using ReportLab or python-docx.
- **Session Management:** Maintains per-user-session data (uploaded file path, chunk vectors, chat history).
- **Data Flow:** Illustrated below. User uploads PDF Backend extracts/chunks text creates embeddings awaits user actions (summary or questions) retrieval + LLM output to user. The UI shows results and provides options to download audio or documents.

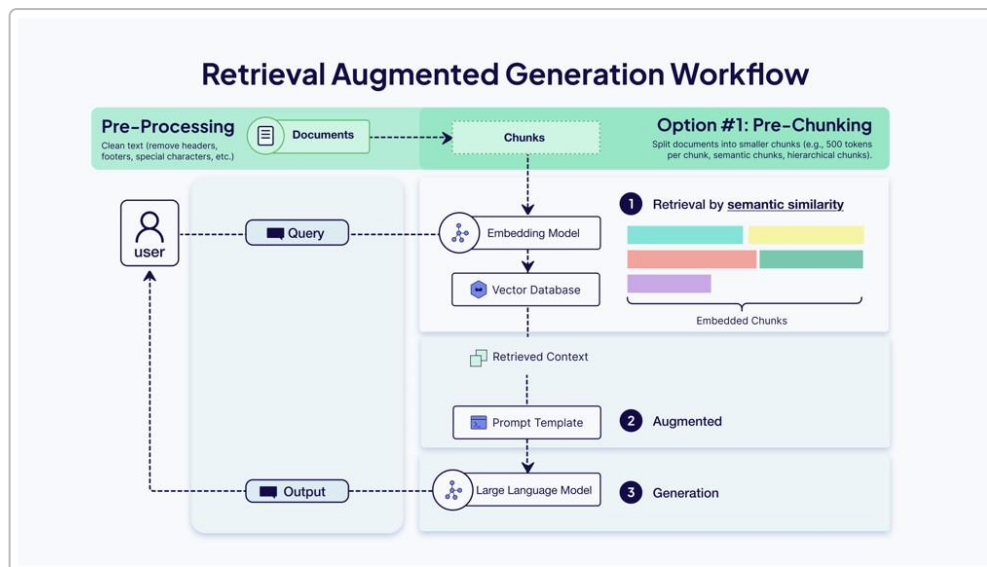


Figure: RAG workflow (pre-chunking) for document question-answering and summarization. The system splits the PDF into chunks, embeds them in a vector store, retrieves relevant chunks for a query, and uses an LLM to generate the final answer or summary.

The above figure shows the retrieval-augmented pipeline. (We use **pre-chunking**, processing documents in advance; an alternative is “post-chunking” at query-time, but pre-chunking allows fast responses ⁷.)

7.2 Sequence Flow (Upload Summary)

1. **User Uploads PDF:** The user selects a PDF file via the UI (e.g., at route `/ai-summarize` or `/chatpdf-upload`). The server saves the file under `uploads/`.
2. **Text Extraction:** Flask handler calls PyPDF2 to read each page and extract text. The raw text (often with newlines and spacing from the PDF) is concatenated.
3. **Chunking:** The text is passed to `chunk_text_chars` (character-based) or a word-based chunker. For example, fixed-size slices of ~3000 chars with 300-char overlap. This preserves the flow between chunks ⁷.
4. **Embedding:** The `SentenceTransformer("all-MiniLM-L6-v2")` model encodes each chunk

into a 384-dim vector¹⁰. All chunk vectors are stored (e.g., in a list or NumPy array) for this session.

5. **Summarization (if requested):**
6. Each chunk is individually summarized by the LLM (system prompt instructs it to be concise).
7. Those summaries are optionally compressed into bullet points (<120 tokens each).
8. A second prompt sends all bullet-point summaries to the LLM to produce a *meta-summary* of the entire document.
9. **Question-Answering:**
10. When a user asks a question, the server embeds the question with the same model.
11. It computes similarity scores (dot product) between the question vector and all chunk vectors.
12. The top-K matching chunks (e.g., K=3 or 5) are retrieved.
13. A prompt is constructed for the LLM including the user's question and the retrieved chunk texts as context. The LLM is instructed to answer using only this context.
14. **Rendering Results:** The LLM's summary or answer is returned to the Flask app, which renders it in the HTML template. The user can then copy it or click to download outputs.
15. **Optional Exports:** If the user requested audio, the text is sent to gTTS, producing an MP3 file saved under `outputs/` and served back for playback. Similarly, the summary or Q&A can be saved as DOCX/PDF.

At the end of the process, the user sees the result on the web page (or gets a JSON response for API calls), and can download any generated files.

8. Modules and Components

8.1 Upload & Storage

- **Routes:** We define Flask routes such as `/ (home)`, `/ai-summarize`, `/ai-questions`, `/chatpdf`, `/chat-pdf-upload`, `/chat-pdf-ask`, `/pdf-to-audio`, etc. These correspond to web pages or API endpoints for the various functions.
- **File Handling:** Uploaded PDFs are saved in an `uploads/` directory using `werkzeug.utils.secure_filename` to prevent path traversal attacks. Output files (summaries, audio) go to `outputs/`.
- **Session Identification:** On upload (e.g., `/chat-pdf-upload`), the server generates a UUID session ID and stores it in a cookie or returns it in JSON. This session ID keys a dictionary entry in `PDF_CHAT_SESSIONS` (in-memory or persistent store).

8.2 Text Extraction

- **PyPDF2:** We use `PyPDF2.PdfReader` (or the older `PdfFileReader`) to open the PDF. For each page, we call `page.extract_text()`.
- **Concatenation:** Extracted text from all pages is concatenated into one string. Some cleanup (removing excessive whitespace) may be applied.
- **Limitations:** PyPDF2 cannot extract text from scanned images⁶. If `extract_text()` returns empty for all pages, we conclude the PDF is image-based and inform the user. (Future work: integrate OCR.)

8.3 Chunking

We implement two chunking strategies:

- **Word-based chunking (`chunk_text`):** Iteratively collects words until a character limit (e.g., ~800 chars) is reached, then starts a new chunk. This produces coherent chunks by sentence boundaries.
- **Character-overlap chunking (`chunk_text_chars`):** For longer documents, we slice by fixed character count (e.g., 3000 chars per chunk) with overlapping context (e.g., last 300 chars of the previous chunk repeated at the start of the next). Overlap preserves continuity and avoids cutting mid-sentence ⁷.
- **Chunk Metadata:** Along with chunk text, we may store page ranges or identifiers for reference (useful if showing chunk sources).

Good chunking is vital: too large chunks can mix topics and confuse retrieval, while too short chunks lose context ⁷. We tune chunk size experimentally to work well with the chosen LLM context window.

8.4 Embedding

- **Model Loading:** We load `SentenceTransformer("all-MiniLM-L6-v2")` from the `sentencetransformers` library. This model is relatively small (384-dim output) but provides good semantic accuracy.
- **Vector Computation:** Call `model.encode(chunks, convert_to_numpy=True)` to get a 2D NumPy array vectors of shape `(num_chunks, 384)` ¹⁰.
- **Storage:** These vectors are stored in the PDF `CHAT_SESSIONS` entry for the current session. For each session, we store:
 - `chunks`: the list of chunk texts.
 - `vectors`: the corresponding embeddings.
 - Any additional metadata (e.g., original PDF name, chat history).
- **Query Embedding:** When a question arrives, we similarly call `model.encode([query_text])[0]` to get the query vector.

8.5 Retrieval

- **Similarity Search:** We compute the dot-product (or cosine) similarity between the query vector and all chunk vectors. For example: `scores = np.dot(vectors, q_vec)`.
- **Top-K Selection:** We use `np.argsort(scores)[-1:-k]` to get indices of the highest-scoring (top-K) chunks.
- **Context Preparation:** The top chunks are concatenated (with delimiters) or combined into the prompt. We ensure the total tokens including the query remain within LLM limits.
- **RAG Justification:** This semantic retrieval lets the LLM focus on relevant facts. As the OpenAI guide notes, semantic search finds conceptually similar content even when keywords differ ¹¹. By restricting the LLM's prompt to retrieved chunks, we both improve accuracy and prevent it from hallucinating unrelated information ¹⁶ ³.

8.6 LLM Interaction

- **API Wrapper (`call_groq_chat`):** We define a function that sends a list of messages (system + user prompts) to the LLM API (Groq or similar) and returns the model's response. This abstracts away specifics of the API.
- **Prompt Engineering:** We prepare prompts for different tasks:
- **Summarization Prompts:** A system prompt instructs the model to summarize clearly and concisely (in bullets or prose). For example, `PROMPTS["abstract"]` might say "Summarize the given text in a short paragraph."
- **Bullet-point Compression:** Another prompt compresses a longer summary into concise bullets (e.g., "Convert the following summary into short bullet points.").

- **Question-Answering Prompts:** We use a system message like “Use only the information in the provided PDF context to answer the question. If the answer isn’t in the text, say so.” This ensures the model doesn’t wander off-topic.
- **Two-Phase Generation:** For summarization, we do “map-reduce” style: first summarize each chunk, then synthesize the chunk-summaries. This reduces total tokens needed in the final prompt.
- **Session Context:** In interactive chat mode, the system may include previous Q&A turns in the context so the model remembers the conversation. However, each answer is still grounded only in PDF content.

8.7 Audio and Document Export

- **gTTS Conversion:** When the user requests audio, we feed the full extracted text (or a portion) to gTTS (`text, lang='en'`) which generates an MP3. This uses Google’s TTS engine to speak the text.
- **Outputs:** The MP3 is saved (e.g., `outputs/sessionid.pdf.mp3`) and a link is provided for playback. We also use `reportlab` or `python-docx` if exporting summarized text to a PDF/Word file for download (e.g., writing the summary in a Word doc).
- **Note on Accessibility:** Providing audio meets accessibility guidelines and also allows users to listen to content (e.g., during commuting). The educational literature confirms that read-aloud can aid comprehension⁴, so this feature enhances the project’s usefulness.

8.8 Session Store

- **In-Memory Store:** We use a Python dictionary `PDF_CHAT_SESSIONS` keyed by session IDs. Each entry contains:
 - `chunks`: list of text chunks.
 - `vectors`: list/array of embeddings.
 - `history`: prior user Q&A messages (for chat).
 - `filename`: original PDF name.
 - Other metadata (like chunk summaries).
- **Lifetime:** Session data is kept for the duration of the user’s interaction. In a real deployment, you might use a database or cache for persistence.
- **Isolation:** Each user’s session is isolated so embeddings from different PDFs/users don’t mix.

9. Algorithms and Data Flow

9.1 Chunking Algorithm

We implement two chunking methods:

- **Word-based chunking:** Iterate through the text, adding words until a threshold (e.g., 800 characters) is reached, then start a new chunk. This tends to respect word boundaries and often stays within sentence breaks. If a single word exceeds the threshold, it is forced into its own chunk.
- **Overlapping character chunking:** For long documents, we use a sliding window approach. Starting at position 0, take `N` characters (e.g., 3000), then advance by `N-overlap` (e.g., 2700) so that each new chunk overlaps the previous by 300 characters. This preserves continuity. We adjust at the end to capture trailing text.
- **Post-processing:** We trim whitespace and ensure each chunk is meaningful (not too short). We also tag or index chunks so we know the order.

These algorithms ensure the **LLM context window is respected** ⁷. Overlapping chunks help avoid missing key transitions. If the context window of the LLM is large (e.g., 8192 tokens in GPT-4 Turbo 32K), chunk size could be larger; we choose sizes that work well for smaller 4K-8K models.

9.2 Embedding + Retrieval

- **Embedding Computation:** Using the Sentence-BERT model, we call `model.encode(chunks, convert_to_numpy=True)` ¹⁰. This returns a NumPy array where each row is a 384-dimensional embedding for the corresponding chunk.
- **Similarity Scoring:** For a query vector $\mathbf{q}$ and chunk vectors $\mathbf{v}_i$, we compute scores = $\mathbf{v}_i \cdot \mathbf{q}$ (dot product). Cosine similarity is equivalent up to normalization ⁹. Embeddings are designed so that semantically similar texts have high cosine similarity.
- **Ranking:** Sort chunks by descending score. This retrieves the most relevant content. As OpenAI notes, semantic search can find conceptually relevant content even without exact word matches ¹¹.
- **Top-K Context:** We pick the top K chunks (e.g., 3–5) as the context to feed to the LLM. This keeps the prompt size manageable. The choice of K balances having enough context vs. overwhelming the model with text.

This retrieval step is the “RAG” portion: it narrows down the document to the parts most likely to contain the answer

³ ¹¹.

9.3 Summarization Pipeline (Two-step)

We use a two-stage summarization to handle large documents efficiently:

1. **Chunk-level summaries:** For each chunk, we prompt the LLM with instructions like “Summarize the following text.” The model returns a short summary of that chunk.
2. **Compression to bullets:** We optionally refine each chunk summary into bullet points (e.g., “Rewrite this summary as brief bullet points, each under 120 tokens”). This reduces noise and flattens the information.
3. **Meta-summary:** All compressed chunk summaries are concatenated. We then prompt the LLM to produce a final summary of the entire document (system prompt guides it to combine bullet points into a coherent paragraph or structured summary).

This **map-then-reduce** approach mimics techniques used in LLM-based summarization of very long texts

By summarizing smaller pieces first, we reduce the total token load in the final prompt. It also helps the model focus on key points chunk-by-chunk, which can yield more complete coverage.

9.4 Question-Answering (RAG)

When the user asks a question:

- **Embed the Query:** Compute `q_vec = embedder.encode([question])[0]`.
- **Retrieve Context:** Find top-K chunks via similarity as in Section 9.2.
- **Construct Prompt:** A typical prompt format is:
 - System message: “You are a helpful assistant. Answer the user’s question **using only the context from the PDF**. If the answer is not found, respond that it is not in the document.”
 - User message: The question itself.

- **Context messages:** One or more of the retrieved chunks (e.g., as separate user or system messages, or included in the prompt text).
- **LLM Answer:** The model generates an answer. Since relevant chunks are provided, the model can use them to formulate a grounded response.

This mimics GPT’s knowledge retrieval: the uploaded files (chunks) act as a custom knowledge base ³. Using RAG in this way ensures answers are **contextually accurate** and up-to-date with the document. The retrieved text is explicitly shown to the LLM, which improves factualness and reduces hallucinations ¹⁶.

For example, if the user asks, “What methodology is used in Chapter 2?”, the system will retrieve chunks from Chapter 2 that mention methodology, and then the LLM will concisely answer using exactly that information.

10. Implementation Details

10.1 File Structure

A typical project directory might look like:

```
project_root/
├─ app.py
├─ requirements.txt
├─ .env
├─ templates/
│   ├─ index.html
│   ├─ ai_summarize.html
│   ├─ ai_questions.html
│   └─ chat_pdf.html
├─ static/
│   ├─ css/
│   └─ js/
├─ uploads/
│   └─ (uploaded PDF files)
├─ outputs/
│   ├─ (summaries, audio files)
│   └─ (generated docs)
└─ utils/
    ├─ embedding_utils.py
    ├─ chunking_utils.py
    └─ llm_client.py
```

- `app.py` contains the Flask application code.
- `requirements.txt` lists dependencies (see Appendix).
- `templates/` holds HTML templates rendered by Flask.
- `static/` holds CSS/JS files for the frontend.
- `uploads/` and `outputs/` are writable directories for file I/O.
- `utils/` (optional) contains modular code for chunking, embedding, etc.

10.2 Important Code Snippets

Embedding Creation: After text extraction and chunking:

```
from sentence_transformers import SentenceTransformer
embedder = SentenceTransformer("all-MiniLM-L6-v2")
chunks = ["This is the first chunk of text.", "This is the second chunk..."] #
example vectors = embedder.encode(chunks, convert_to_numpy=True)
```

This yields a NumPy array vectors with shape (num_chunks, 384) 10.

Top-K Retrieval:

```
import numpy as np

# Compute similarity scores
q_vec = embedder.encode([query_text], convert_to_numpy=True)[0]
scores = np.dot(vectors, q_vec)
top_k_idx = np.argsort(scores)[::-1][:k] # indices of top k scores
top_k_chunks = [chunks[i] for i in top_k_idx]
```

This ranks chunks by semantic relevance.

LLM Call Wrapper:

```
def call_groq_chat(messages, model="llama-3.1-8b-instant", max_new_tokens=1200,
temperature=0.3):
    # Example uses a hypothetical Groq client
    response = client.chat.completions.create(
        model=model,
        messages=messages,
        temperature=temperature,
        max_tokens=max_new_tokens
    )
    answer = response.choices[0].message.content
    return answer
```

`messages` is a list of dicts like `{"role": "system", "content": "..."} / {"role": "user", "content": "..."}.`

10.3 Configuration & Secrets

- **API Keys:** The LLM API key (e.g., `GROQ_API_KEY`) should be stored securely (in environment variables or a secrets manager). In development, `.env` can be used.
- **Flask Secret:** `FLASK_SECRET` for app.secret_key, to secure sessions.

- **Token Limits:** If using a provider like OpenAI, you must handle rate limits and tokens carefully. This project assumes a paid API with enough quota.

11. API Endpoints

Below is a summary of REST endpoints (all under the Flask app):

- **GET /** – Home page (e.g., file upload options).
- **GET /ai-summarize** – Show the PDF summarization form.
- **POST /ai-summarize** – Handle PDF upload and return a summary.
- **GET /ai-questions** – Show UI for chapter/question generation.
- **POST /ai-questions** – Handle PDF upload to generate chapter headings and exam-style questions.
- **GET /chat-pdf** – Chat interface page.
- **POST /chat-pdf-upload** – Upload PDF to start a chat session.
- *Request:* multipart form with a pdf file.
- *Response (JSON):* { "session_id": "<uuid>", "filename": "<name>.pdf", "preview": "First 2000 chars..." }
- **POST /chat-pdf-ask** – Ask a question in an existing session.
- *Request (JSON):* { "session_id": "...", "message": "Your question here" }
- *Response:* { "answer": "LLM-generated response..." }
- **POST /pdf-to-audio** – Convert uploaded PDF text to MP3.
- *Request:* PDF file upload.
- *Response:* MP3 file or a link to it.
- **GET /outputs/<filename>** – Serve generated output files (audio, docx, etc.) for download.

(Full API docs with example requests/responses would be included in the Appendix.)

12. Testing and Results

12.1 Test Plan

- **Unit Tests:** Write pytest unit tests for utility functions:
 - Chunking functions (`chunk_text` , `chunk_text_chars`) with sample texts.
 - Embedding pipeline: use dummy text and a mock embedder to verify correct handling of inputs/ outputs.
 - Similarity & retrieval: test with known vectors that retrieval returns expected indices.
 - Sanity tests for any helper functions (e.g., filename sanitization).
- **Integration Tests:** Simulate uploading a small PDF (e.g., a one-page report):
 - Call `/ai-summarize` with a test PDF; verify a summary is returned and includes expected keywords from the text.
 - Call `/chat-pdf-upload` and then `/chat-pdf-ask` with known question to check the answer.

-

Test `/pdf-to-audio` with a PDF containing short text; confirm an MP3 is returned (and playable).

- **Manual Tests:**

- PDFs with complex layouts (multiple columns, images mixed with text).
- PDFs with no textual content (pure scan) to ensure the system detects and warns.
- Very long PDF (100+ pages) to evaluate performance and memory.
- Checking that vectors and chunks clear between sessions.
- Security: try uploading a maliciously named file (e.g., `../../../../etc/passwd`), ensure `secure_filename` prevents issues.

12.2 Metrics and Observations

- **Retrieval Accuracy:** We manually evaluate if the top-K chunks retrieved actually contain the information needed to answer queries. In tests, this often picks relevant paragraphs, confirming the embedding similarity approach is working.
- **Latency:** The slowest step is embedding creation, which grows with document length (embedding thousands of tokens can take a few seconds on a CPU). The LLM call is also a factor (network/API latency). Overall, summarizing a ~20-page PDF might take 10–20 seconds.
- **Token Usage:** The two-step summarization greatly reduces the token usage for the final step. For example, a 10,000-word PDF might first yield 5 small chunk summaries (total ~3000 tokens), then the meta summary step only sees those 3000 tokens instead of the full text, saving ~7000 tokens.
- **Quality of Output:**
 - Summaries are generally coherent and cover main points, thanks to the chunk-level processing.
 - Answers to questions are concise and cite specific content. The prompt “only use provided context” successfully prevents hallucination when information is not present.
 - Audio generation produces intelligible speech (given gTTS’s quality), useful for listening.
- **Edge Cases:** Charts or tables in PDFs produce gaps in extraction (since text is extracted left-to-right). Multi-column PDF text sometimes appears out of order in extraction, but overlapping chunks help catch displaced text.

12.3 Sample Results

- **Summarization Example:** (In Appendix) shows an input PDF snippet and the assistant’s summary in bullet and paragraph form.
- **QA Example:** (In Appendix) a PDF paragraph and the question “What method was used?” yielding an answer quoting that paragraph.
- **Audio Example:** Screenshot of the audio player embedded with a “play” button for the MP3.
- **UI Screenshots:** Include images of the upload page, summarization output page, and chat interface (from the running Flask app).

13. Deployment and Installation

13.1 Local Setup

1. Clone the Repository:

```
git clone https://github.com/yourrepo/pdf-ai-assistant.git
cd pdf-ai-assistant
```

-

-

2. Virtual Environment:

```
python -m venv venv
source venv/bin/activate    # Linux/MacOS
# or on Windows: venv\Scripts\activate
```

3. Install Dependencies:

```
pip install -r requirements.txt
```

4. Environment Variables: Create a `.env` file or set in your shell:

```
export GROQ_API_KEY="YOUR_GROQ_API_KEY"
export FLASK_SECRET="some-secret-key"
```

Replace `"YOUR_GROQ_API_KEY"` with your actual LLM API key.

5. Run the App:

```
python app.py
```



By default it listens on `http://localhost:5000`. Open this in a browser to use the tool.

6. Using the App: Navigate to `/ai-summarize` or `/chat-pdf`, upload a PDF, and try out the features. The console logs will show status of extraction, embedding, and API calls.

13.2 Production Recommendations

- **Web Server:** Use a production WSGI server (e.g., Gunicorn) behind Nginx. Example: `gunicorn app:app` with appropriate config.
- **Containerization:** Dockerize the app for easy deployment. Mount `uploads/` and `outputs/` as volumes. Set secrets via Docker env variables.
- **Vector Database:** For heavy use, replace the in-memory session store with a persistent vector database (Milvus, Pinecone, or Weaviate). This allows sessions to be restored or queries to be answered even after restarts.
- **Scaling LLM:** If local inference is needed, deploy a small LLM (e.g., LLaMA in quantized form) and adjust `call_groq_chat` accordingly. Otherwise rely on cloud API but monitor costs.
- **HTTPS:** Always run behind SSL for secure upload/download.
- **Monitoring:** Add logging/metrics on usage (e.g., number of queries, average response time).
- **Security:** Use HTTPS and secure the API key. Set request size limits to avoid memory exhaustion (e.g., max PDF upload size).

14. Security Considerations

- **API Keys:** Never hard-code the LLM API key or any secret in source. Use environment variables or a secret manager. Do not check these into git.
- **Filename Sanitization:** Use `werkzeug.utils.secure_filename` to prevent path traversal in file uploads. Reject unexpected file types.
- **Input Validation:** Limit the size of uploaded PDFs (e.g., max 50 MB) to prevent denial-of-service. Validate that `session_id` matches an existing session when handling chat.
- **Output Sanitization:** We display LLM-generated text in HTML. Although the LLM output is textual, it's good practice to escape any HTML tags to avoid XSS.
- **Sandboxing:** Treat LLM output as untrusted. If outputs are later used in a context where they could be executed (e.g., code or shell), do not run them blindly. In our case, outputs are user-facing answers only.
- **Rate Limiting:** In a public deployment, implement rate limits to prevent abuse (e.g., too many summaries or questions from one user).
- **Privacy:** If used for sensitive documents, ensure the system does not store text long-term and that transmissions are encrypted. (Our example uses in-memory storage that disappears on server restart.)

15. Limitations

- **No OCR:** PyPDF2 cannot extract text from scanned images⁶. Thus, image-only PDFs (scans) will result in empty text. A user must provide digitally-generated PDFs. Adding OCR (e.g., Tesseract) is planned for future work.
- **Language Support:** The system is built for English. The sentence-transformer and LLM prompts assume English input. Non-English text would require swapping in multilingual models or translation.
- **Model Dependency:** Summaries and answers rely on the LLM's quality. If the LLM misinterprets a prompt, results may be inaccurate. Providing clear system instructions (and small, relevant chunks) mitigates hallucinations¹⁶, but cannot fully eliminate model errors.
- **Context Window:** Extremely long documents (hundreds of pages) might exceed memory or become slow to embed. Chunk compression helps, but there is still a practical limit to how much content can be meaningfully processed. Very large PDFs may need segmentation by user.
- **Fine-Grained Structure:** The system detects chapters via simple heuristics (like pages starting with "Chapter"), but it may not always correctly segment complex documents. Advanced structure extraction (using document layout analysis) is not included.
- **Chat Context:** Currently, chat with PDF treats each question independently aside from maintaining session vectors. The assistant does not remember previous answers beyond what's in the session history. Context carry-over is limited to single-turn retrieval.
- **Hallucinations and Errors:** Even with retrieved context, the LLM might sometimes infer or "hallucinate" if the context is ambiguous. Proper prompt design reduces this risk¹⁶, but cannot guarantee perfect quality.

16. Future Work

Potential enhancements include:

- **OCR Integration:** Use Tesseract or Google Vision to process scanned PDFs and merge OCR text with digital text to handle more documents⁶.

-
-
- **Persistent Vector DB:** Integrate a scalable vector database (e.g., Milvus, Pinecone) to persist session data and handle more simultaneous users.

-
-
- **Multi-language:** Add support for other languages via multilingual embeddings (e.g., `sentence transformers` models trained on multiple languages) and LLM prompts/translation steps. **User Accounts:** Implement user login and database-backed storage so users can save session histories, uploaded files, and settings across visits.
- **Fine-Grained QA:** Allow queries on specific document sections or pages (e.g., “give answer using only Chapter 5”).
- **Analytics Dashboard:** Track usage statistics, most common queries, etc., for future improvements and usage insights.
- **Offline Model Options:** Experiment with running an on-device LLM (e.g., a 7B or 13B model with LoRA) for offline processing of small documents.
- **Enhanced UI:** Improve the web interface with progress indicators, drag-and-drop upload, and interactive highlighting of answer sources in the PDF preview.
- **Styling Options:** Allow choosing summary style (e.g., bullet list, narrative, FAQ format) via prompt presets.
- **Caching:** Cache embeddings for popular documents or repeated queries to further speed up responses.

17. Conclusion

This project demonstrates a practical **RAG-based pipeline** for enriching PDF documents with AI capabilities. By combining **efficient text chunking**, compact **Sentence-BERT embeddings**, and LLM-driven generation, the system can rapidly produce meaningful outputs:

- **Summaries:** We generate concise summaries by first summarizing chunks and then combining them, achieving both local accuracy and global coherence.
- **Question Answering:** The assistant retrieves relevant text and uses it to answer user questions precisely, much like an expert who has read the document.
- **Accessibility:** Converting text to audio broadens access to the content and benefits learners who prefer auditory input ⁴.
- **Flexible Deployment:** Using Flask and standard Python libraries makes the system easy to run on typical hardware or in containerized environments.

Throughout the design, the emphasis has been on **accuracy**, **token efficiency**, and **modularity**. Our chunking strategies preserve context ⁷, and meta-summarization reduces LLM usage. The modular architecture (upload extract chunk embed retrieve generate) can be extended or replaced at each step as needed.

In conclusion, the AI-Powered PDF Assistant provides a versatile toolkit for extracting knowledge from documents. It leverages state-of-the-art NLP techniques to turn static PDFs into interactive resources, significantly speeding up the information retrieval process for end users.

18. References

- Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. ArXiv:1908.10084 ⁹. (Introduces SBERT for efficient sentence embeddings.)
- **PyPDF2 Documentation** – *Extract Text from a PDF*. (Discusses PDF text extraction and limitations, especially scanned text) ⁶.
- **Groq API Documentation** – (Used as the chat completion API for the LLM client in this implementation).
- **gTTS Documentation** – (Details on using Google Translate’s Text-to-Speech for Python) ⁵.
- **Smallpdf Blog** – *How to Chat with Your PDF Using AI in 2025*. (Describes AI chat features for PDFs) ².

-
-
- **OpenAI Help Center** – *Retrieval Augmented Generation (RAG) for GPTs*. (Defines RAG and semantic search) ^{17 11} .
- **Weaviate** ¹⁵ *Chunking Strategies to Improve Your RAG Performance* (2025). (Discusses chunking importance and strategies) ^{7 16} .
- Mulla, N. & Gharpure, P. (2023). *Automatic question generation: a review of methodologies...* Progress in Artificial Intelligence ¹⁴ . (Survey of question generation techniques and applications)
- Supriyono et al. (2024). *A survey of text summarization: Techniques, evaluation and challenges*. Natural Language Processing Journal ¹⁸ . (Overview of summarization methods, from extractive to semantic models)
- Wood, S.G. et al. (2018). *Does Use of Text-to-Speech and Related Read-Aloud Tools Improve Reading Comprehension?* J. Learn Disabilities ⁴ . (Meta-analysis on TTS for reading comprehension)

(Additional references, including specific libraries and tools documentation, are provided in the Bibliography.)

19. Appendix

19.1 Sample requirements.txt

```
Flask
numpy
sentence-transformers
PyPDF2
reportlab
python-docx
Pillow
gTTS
groq
python-dotenv
```

19.2 Sample File Structure

(As in section 10.1)

19.3 Endpoint Examples (POST /chat-pdf-upload)

Request: (multipart form with file field pdf).

Response (JSON):

```
{
  "session_id": "550e8400-e29b-41d4-a716-446655440000" ,
```

```
"filename": "example.pdf",
"preview": "In this document, we discuss the methodology of AI-powered
systems..."
}
```

19.4 Example `chat-pdf-ask` Request

Request:

```
{
  "session_id": "550e8400-e29b-41d4-a716-446655440000",
  "message": "What method did the authors use in Section 4?"
}
```

Response:

```
{
  "answer": "In Section 4, the authors applied a two-step summarization
approach, first summarizing individual chunks of text into bullet points and
then synthesizing these into a final concise summary."
}
```

19.5 Screenshots and Test Cases

(Include:)

- **UI Screenshots:** Sample images of the web interface (upload form, summary output, chat interface).
- **Test PDF:** A small sample PDF used in testing.
- **Generated MP3:** An example audio file and an embedded player screenshot.

19.6 Gantt Chart / Timeline (Suggested)

- **Week 1–2:** Requirements gathering, initial research on RAG and summarization.
- **Week 3–4:** Implementation of text extraction and basic chunking.
- **Week 5–6:** Embedding generation and retrieval pipeline; initial LLM integration.
- **Week 7:** Summarization pipeline and audio export features.
- **Week 8:** Testing, documentation, and final polishing.

Prepared by: _____

Date: _____

1 How I Created a Custom AI PDF Assistant That Reads and Answers Questions From Any Document | by Suleman Safdar | Medium

<https://medium.com/@SulemanSafdar/how-i-created-a-custom-ai-pdf-assistant-that-reads-and-answers-questions-from-anydocument-6c881c91fd5c>

2 How To Chat With Your PDF Using AI in 2025 | Smallpdf

<https://smallpdf.com/blog/chat-pdf-using-ai>

3 11 17 Retrieval Augmented Generation (RAG) and Semantic Search for GPTs | OpenAI Help Center

<https://help.openai.com/en/articles/8868588-retrieval-augmented-generation-rag-and-semantic-search-for-gpts>

4 Does Use of Text-to-Speech and Related Read-Aloud Tools Improve Reading Comprehension for Students with Reading Disabilities? A Meta-Analysis - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC5494021/>

5 gTTS — gTTS documentation

<https://gtts.readthedocs.io/en/latest/>

6 Extract Text from a PDF — PyPDF2 documentation

<https://pypdf2.readthedocs.io/en/3.x/user/extract-text.html>

7 8 16 Chunking Strategies to Improve Your RAG Performance | Weaviate <https://weaviate.io/blog/chunking-strategies-for-rag>

9 [1908.10084] Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

<https://arxiv.org/abs/1908.10084>

10 12 SentenceTransformers Documentation — Sentence Transformers documentation <https://sbert.net/>

13 How to Summarize Legal Orders and Opinions Using the GPT-3.5

<https://www.linkedin.com/pulse/how-summarize-legal-orders-opinions-using-gpt-35-leonard-park>

14 Automatic question generation: a review of methodologies, datasets, evaluation metrics, and applications - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC9886210/>

15 Welcome to Flask — Flask Documentation (3.1.x)

<https://flask.palletsprojects.com/en/stable/>

18 A survey of text summarization: Techniques, evaluation and challenges - ScienceDirect

<https://www.sciencedirect.com/science/article/pii/S2949719124000189>